How Well Do COCO Detectors Drive? Zero-Shot Transfer to KITTI, Speed, and Fine-Tuning

Aaron Chandler · CMPE 249, San José State University · HW1, Track A (experimental) · November 2026
Code, configuration and command log: `github.com/AaronChandler976/CMPE_249_HW1`

Summary. Five COCO-pretrained detectors from four families (YOLO11s, RT-DETR-L, Faster R-CNN R50-FPN v2, FCOS R50-FPN, and the open-vocabulary YOLO-World-S) were evaluated on COCO val2017 and, without retraining, on KITTI val (1,496 images) through a documented KITTI→COCO label mapping. A YOLO11s fine-tuned on KITTI train serves as the in-domain reference. The pipeline first reproduces every model's published COCO mAP to within 1.4 AP. On KITTI, AP on the shared classes (person, car, truck) falls by 12.8–23.6 points. The drop is *class-specific*: car is essentially unchanged while person falls 33–44 points. It occurs at all object scales. Much of the truck loss comes from KITTI vans being labelled "truck". The 16-point COCO spread between models shrinks to 4.4 points on KITTI. Twenty-five minutes of fine-tuning raises KITTI AP from 24.6 to 71.6. All five models run in real time at batch 1 on an RTX 5070 Ti (5–19 ms per KITTI image).

1. Question and approach

Detectors are usually compared on COCO, but a robot sees a road, not COCO. The question is: *how much of a COCO-trained detector's skill transfers to a driving dataset, which classes and object sizes break, and is the fix a better architecture or in-domain data?* The approach is a controlled experiment with one evaluation harness (pycocotools, identical thresholds) for every model. The harness was validated by reproducing known COCO numbers before any KITTI number was trusted, and automated sanity checks gate every result (109 PASS / 0 FAIL). Four analyses follow: cross-dataset generalization by class and scale, architecture plus speed–accuracy, closed- vs open-vocabulary detection, and fine-tuning. A resolution ablation, a failure analysis and four targeted verification experiments complete the study.

2. Models and datasets

Model	Family / key idea	Checkpoint	Input	Params (M)	Post-proc.	Published COCO mAP
YOLO11s [13]	One-stage, anchor-free CNN	yolo11s.pt (Ultralytics)	640	9.5	NMS	47.0 [13]
RT-DETR-L [8]	Transformer decoder, set prediction (DETR [7])	rtdetr-l.pt	640	33.0	none (1-to-1 matching)	53.0 [14]
Faster R-CNN v2 [3,4,5]	Two-stage, anchor-based, FPN	torchvision COCO_V1	short side 800	43.7	NMS	46.7 [16]
FCOS [6]	One-stage, anchor-free, per-pixel + centerness	torchvision COCO_V1	short side 800	32.3	NMS	39.2 [16]
YOLO-World-S v2 [9]	Open-vocabulary, CLIP text prompts [10]	yolov8s-worldv2.pt	640	12.8	NMS	37.7 (zero-shot) [15]
YOLO11s-KITTI	YOLO11s fine-tuned on KITTI train	own run (best.pt)	640	9.4	NMS	–

Table 1. Models: four detector families (two-stage, dense one-stage, DETR-style, open-vocabulary) plus one fine-tuned model. Params are counted from the loaded model. torchvision models resize to a short side of 800 px (max 1,333).

Datasets. *COCO val2017* [1]: 5,000 images, 36,781 boxes, 80 classes. *KITTI 2D object* [2], Ultralytics release [18]: the 7,481 labelled training images split into 5,985 train / 1,496 val. All KITTI numbers below are on this val split, which was never used for training. The release drops KITTI's *DontCare* regions, which matters in Section 7. KITTI val holds 7,920 boxes, of which car makes up 72% (5,716). Under COCO's absolute area thresholds (32^2/96^2 px), KITTI cars are 17% small, 53% medium and 30% large.

Taxonomy mapping. COCO models predict COCO classes, so KITTI ground truth was relabelled (Table 2). This follows the KITTI devkit's "neighbouring class" idea: classes with no clean COCO counterpart become *ignore regions* (`iscrowd=1`). A

prediction on an ignore region is neither rewarded nor punished. Dropping such a class instead would turn every correct "car" box on a van into a false positive. Only person, car and truck are scored, and every COCO→KITTI comparison uses **the same three classes on COCO** ("COCO 3-cls AP"), never the 80-class mAP.

KITTI class	COCO class	Role	Val boxes	Reason
Car / Truck / Pedestrian	car / truck / person	scored	5,716 / 223 / 921	same concept
Van	car	ignore region	555	COCO has no van; models call vans car or truck
Person_sitting, Cyclist	person	ignore region	36, 352	KITTI cyclist box = rider + bike; a COCO person box covers only the rider
Tram, Misc	–	dropped	117, –	no COCO equivalent / undefined

Table 2. KITTI→COCO mapping (`configs/experiments.yaml: taxonomy`, `docs/taxonomy_mapping.md`). 6,860 boxes are scored and 943 are ignore regions. Models that output KITTI labels (YOLO-World with KITTI prompts, the fine-tuned model) map van→car and person_sitting→person, and are also scored on all 7 native classes.

3. Experimental setup and hardware

Hardware/software. NVIDIA GeForce RTX 5070 Ti (16 GB, driver 617.42), AMD CPU, Windows 11 (WDDM driver model); Python 3.12.16, PyTorch 2.11.0+cu128, torchvision 0.26.0, CUDA 12.8, cuDNN 9.19, Ultralytics 8.4.197, pycocotools 2.0.11. Full versions are captured automatically in every `results/run_meta/*.json`.

Inference and evaluation. All models use conf = 0.001, NMS IoU = 0.7 (not used by RT-DETR) and max 300 detections per image. A low confidence threshold is required for mAP: a deployment threshold such as 0.25 would cut the precision–recall curve short and lower AP. Metrics are COCO-style AP@[.5:.95] (the mean of AP at IoU 0.50, 0.55, …, 0.95, each AP being the area under a 101-point interpolated precision–recall curve), AP50, AP75, $AP_{S/M/L}$, AR100 and per-class AP, all from pycocotools [17]. Inference is FP16 except FCOS, which runs in FP32 (Section 8). Qualitative and failure analysis uses conf = 0.30.

Latency protocol. Batch 1, images preloaded in RAM, end-to-end time (preprocess + network + NMS) with `torch.cuda.synchronize()`. There are 200 untimed warm-up images, so every input shape has been seen once, followed by 3 timed passes over 200 images. The reported latency is the median of the three pass means, with the min–max range. Peak memory is `torch.cuda.max_memory_allocated`.

Fine-tuning. YOLO11s initialised from COCO weights, 50 epochs, image size 640, batch 16, early-stop patience 15 (not triggered), seed 0, deterministic, Ultralytics default optimiser and augmentation. Training took 0.41 h. The best checkpoint by Ultralytics validation was the last epoch (50), so selecting it on the val split gave no extra advantage.

4. Commands and configuration

One config file (`configs/experiments.yaml`) defines every model, threshold, mapping and run. One script runs the stages, and every run writes a metadata file with its exact command, thresholds, image count and environment. The complete command log, including diagnostics, is `commands.md`.

```
python run_all.py --quick                                    # 2.5-min smoke test (25 images/run, 1-epoch fine-tune)
python run_all.py --stages prepare                           # COCO val2017 + KITTI, builds COCO-format KITTI GT
python run_all.py --stages predict evaluate                  # 5 models x {COCO, KITTI} + YOLO11s@1280 (14 min)
python run_all.py --stages finetune predict_ft evaluate      # fine-tune YOLO11s on KITTI (29 min)
python run_all.py --stages benchmark failures analyze sanity # latency/memory, error analysis, tables, checks
# per-run equivalent, e.g.:  python scripts/predict.py --model yolo11s --dataset kitti --imgsz 1280 --tag res1280
#                            python scripts/evaluate.py --run yolo11s__kitti --target kitti_cocomap
```

5. Results

Model	COCO AP (published)	COCO AP (ours)	COCO AP50	COCO 3-cls AP	KITTI AP	KITTI AP50	KITTI AP75	KITTI AP_S	KITTI AP_L	KITTI AR100
YOLO11s	47.0	45.6	62.3	46.4	24.6	48.2	22.5	14.0	36.8	42.7
RT-DETR-L	53.0	**52.8**	71.2	**52.6**	**29.0**	53.8	27.4	22.4	40.9	**55.6**
Faster R-CNN v2	46.7	47.0	68.1	50.8	28.3	54.4	27.3	15.3	40.0	54.7
FCOS	39.2	39.3	58.7	43.4	28.7	**54.5**	27.4	21.0	38.5	53.8
YOLO-World-S v2	37.7	36.8	51.3	41.6	28.8	50.0	29.5	**23.7**	**42.4**	54.0
YOLO11s @1280	–	–	–	–	26.9	51.0	25.1	21.9	36.3	47.0
YOLO11s-KITTI (fine-tuned)	–	–	–	–	**71.6**	**93.0**	**80.3**	**58.3**	**80.7**	**77.1**

Table 3. Main results (AP@[.5:.95] in %, unless noted). COCO: 80 classes, 5,000 images. KITTI: person/car/truck, 1,496 val images, zero-shot except the last row. Every published COCO mAP is reproduced to within 1.4 AP, which validates id mapping, coordinates and thresholds before any KITTI number is interpreted. Bold = best zero-shot / best overall in a column. Source: `results/tables/main_results.md`, `results/sanity_report.md`.

6. Analyses

6.1 Cross-dataset generalization: which classes and scales degrade?

Model	COCO 3-cls	KITTI 3-cls	Drop	person		car		truck	
				COCO	KITTI	COCO	KITTI	COCO	KITTI
YOLO11s	46.4	24.6	−21.8	58.8	19.5	44.9	45.3	35.5	9.0
RT-DETR-L	52.6	29.0	−23.6	61.7	20.1	50.0	49.5	46.1	17.4
Faster R-CNN v2	50.8	28.3	−22.5	59.6	20.5	50.0	46.4	42.8	18.1
FCOS	43.4	28.7	−14.7	53.5	20.2	42.4	46.4	34.3	19.7
YOLO-World-S	41.6	28.8	−12.8	56.5	12.8	39.2	45.9	29.2	27.7

Table 4. Domain shift on the shared classes (AP@[.5:.95]). Source: `results/tables/domain_shift.md`.

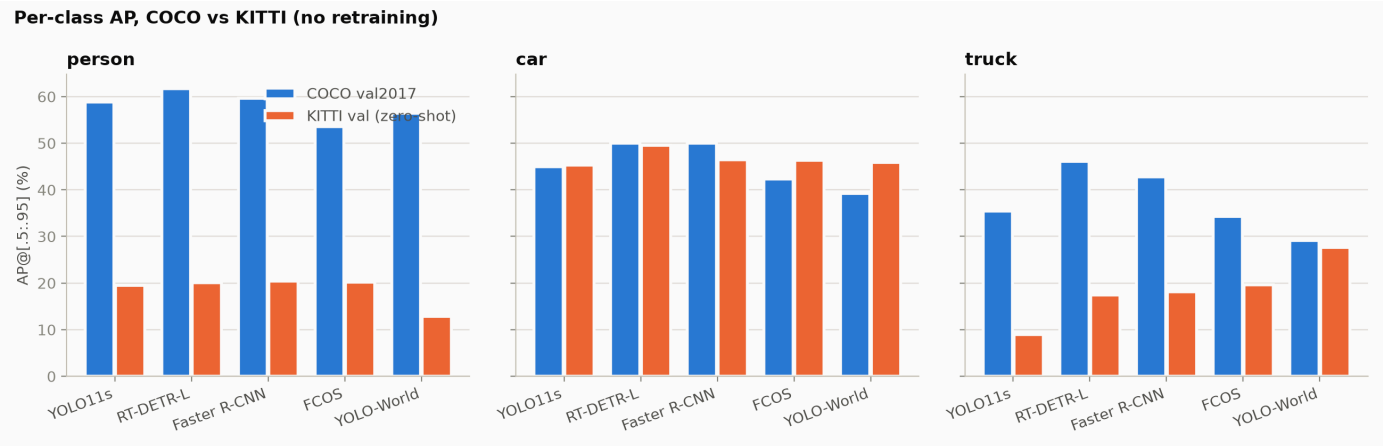


Figure 1. Per-class AP on COCO (blue) and KITTI (orange). Car transfers, while person collapses for every architecture and truck for the closed-set models.

The shift is class-specific, not uniform. Car AP is essentially unchanged (−3.6 to +6.7), but person loses 33–44 points for *every* model and truck loses up to 29. A road scene is therefore not "harder" across the board. What breaks is the definition and appearance of particular classes. KITTI pedestrians are small, upright and often occluded, whereas COCO "person" covers every pose, crop and context. The AP gap also widens at stricter IoU. On COCO, AP75/AP50 is 0.79–0.80

for RT-DETR and YOLO11s, but on KITTI it is about 0.47–0.51 for the closed-set models, and back to 0.86 after fine-tuning. The zero-shot models often *find* the object but draw a box that does not match KITTI's annotation convention.

Truck and vans (a mapping side effect). Of the confident (score ≥ 0.3) "truck" boxes, 48% (YOLO11s), 34% (RT-DETR) and 38% (Faster R-CNN) lie on a KITTI *van*. Only 13–20% hit a real truck. Because pycocotools applies ignore regions per category, and vans are ignored for *car* only, each such box is a truck false positive. A sensitivity check (Table 5) also marks vans as ignore regions for truck. Truck AP then rises by 5.7–11.1 points for every zero-shot model, while the fine-tuned model is unaffected because it learned the van/truck distinction. Van confusion is therefore a large part of the truck drop, but not all of it: RT-DETR still loses 17.6 points (46.1→28.5). The main mapping was not changed after seeing results; Table 5 is reported only as a sensitivity analysis.

Variant (KITTI val)	YOLO11s	RT-DETR-L	Faster R-CNN	FCOS	YOLO-World	Fine-tuned
Main mapping: truck AP / 3-cls AP	9.0 / 24.6	17.4 / 29.0	18.1 / 28.3	19.7 / 28.7	27.7 / 28.8	83.9 / 71.6
Van also ignored for truck: truck / 3-cls	14.7 / 26.5	28.5 / 32.7	25.0 / 30.6	27.9 / 31.5	36.9 / 31.9	83.9 / 71.6
Van scored as car: car AP (main → variant)	45.3→43.8	49.5→48.1	46.4→44.6	46.4→45.3	45.9→45.3	80.9→80.8

Table 5. Taxonomy sensitivity, re-scoring the same predictions (`results/tables/sensitivity_taxonomy.md`; box counts in `results/metrics/diagnostics/truck_on_van.txt`). Treating vans differently moves car AP by only 0.1–1.8, so ignoring vans is *not* why some models score higher on KITTI cars than on COCO cars.

Scale. A common expectation is that KITTI is hard because its objects are small. The data only partly support this (Figure 2). Large objects lose almost as much as small ones (YOLO11s 69.6→36.8 large vs 27.9→14.0 small), and most KITTI ground truth is *medium*. Doubling the input resolution of YOLO11s to 1280 (Table 6) raises AP_S by 7.9 points, but leaves AP_L flat and recovers only 2.3 AP overall. Pixels help small objects, but most of the gap lies elsewhere.

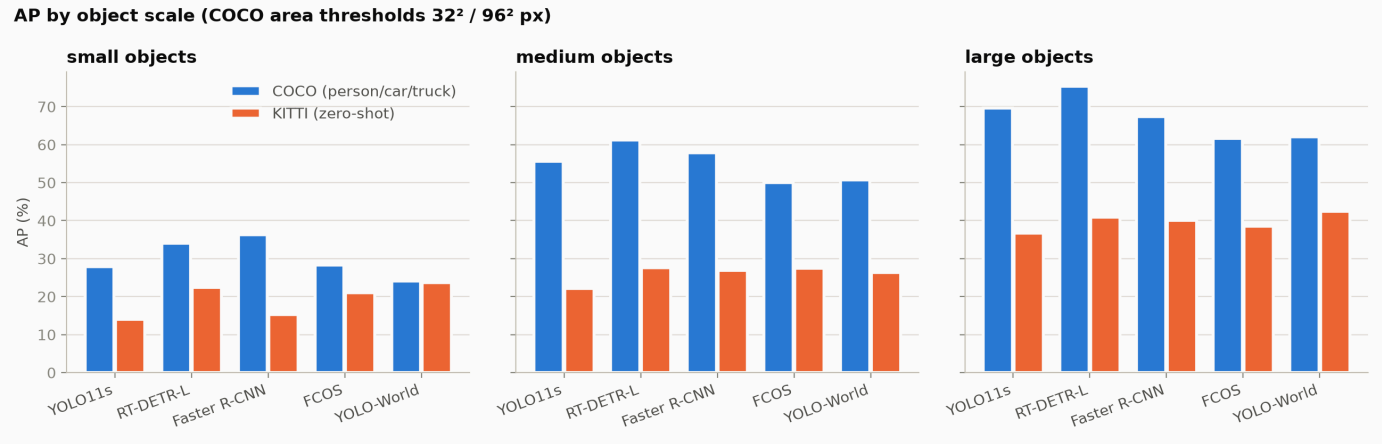


Figure 2. AP by COCO area bucket on the three shared classes. The drop is large at every scale (`results/tables/domain_shift_scale.md`).

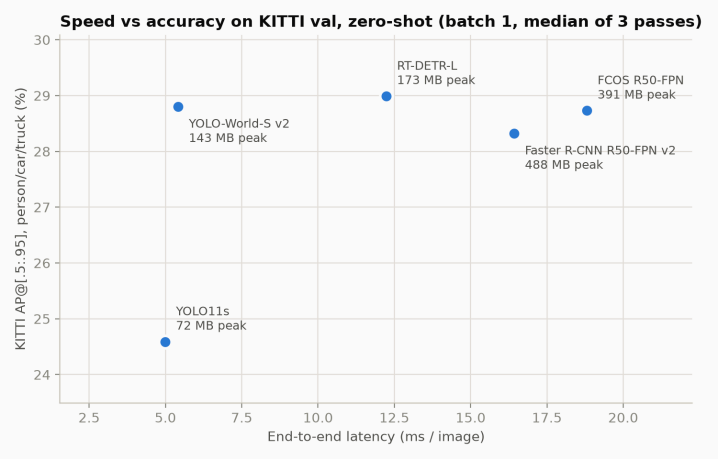
YOLO11s on KITTI	AP	AP50	AP_S	AP_M	AP_L	AR100	Latency (ms)	Peak mem (MB)
imgsz 640	24.6	48.2	14.0	22.2	36.8	42.7	5.0	72
imgsz 1280	26.9	51.0	21.9	25.2	36.3	47.0	5.6	79

Table 6. Resolution ablation. Since KITTI images are 1242×375, 640 px halves every object. At 1280 they are near native size. The model was trained at 640, so large objects at 1280 fall outside its training scale range (`results/tables/resolution.md`).

6.2 Architecture comparison and speed–accuracy trade-off

Model	KITTI latency ms (range)	FPS	Peak mem MB	KITTI AP	COCO latency ms (range)	COCO peak MB	COCO AP
YOLO11s	**5.0** (5.0–5.0)	200	72	24.6	7.2 (6.9–7.6)	71	45.6
YOLO-World-S v2	5.4 (5.4–5.4)	185	144	28.8	6.2 (6.2–6.4)	81	36.8
RT-DETR-L	12.2 (12.2–12.3)	82	173	29.0	16.7 (13.7–18.8)	143	52.8
Faster R-CNN v2	16.4 (16.2–16.6)	61	488	28.3	19.4 (19.3–19.7)	637	47.0
FCOS (FP32)	18.8 (18.7–19.2)	53	391	28.7	25.6 (25.5–26.2)	501	39.3

Table 7. Speed and memory, batch 1, median of 3 passes × 200 images (`results/tables/benchmark.md`). Latency is end-to-end and includes each framework's own preprocessing and input size, so it compares deployable pipelines rather than bare networks. FCOS runs FP32, the others FP16.



Speed. On this GPU every model is real-time (>50 FPS). The one-stage YOLOs are 3.0–3.8× faster than the R50-FPN models, which also take an 800-px input and 2.7–6.8× the peak memory.

Accuracy rankings do not transfer. The COCO spread of 16 AP (36.8–52.8) shrinks to 4.4 AP on KITTI. YOLO-World is *last* on COCO and ties for *first* on KITTI, while YOLO11s is mid-pack on COCO and last on KITTI. On KITTI, YOLO-World matches RT-DETR (28.8 vs 29.0) at less than half the latency, so it is on the speed–accuracy frontier together with YOLO11s.

NMS vs NMS-free. RT-DETR has the best zero-shot KITTI AP and AR100 (55.6), plausibly because one-to-one query matching does not suppress overlapping neighbours in rows of parked cars. Differences of 1–2 KITTI AP are within the uncertainty of a single val split, however, so the architecture ranking on KITTI is a near-tie.

Figure 3. KITTI latency vs KITTI AP. KITTI latencies are used because they are stable to ±0.5 ms across passes. COCO latencies vary by up to 5 ms on Windows/WDDM with variable input shapes.

6.3 Closed-set vs open-vocabulary detection

YOLO-World takes class names as text and turns them into classifier weights with a CLIP text encoder. This lets it be prompted with KITTI's own seven classes and scored natively, which a closed-set COCO model cannot do. It transfers best overall (smallest drop, −12.8; best truck and small-object AP). But it is weakest on person (12.8), and on native KITTI classes it reaches only 15.3 AP: pedestrian 7.1, cyclist 2.2, person_sitting 0.3, tram 7.4. **The prompt word matters.** Replacing "pedestrian" with "person", with identical weights, raises person AP from 12.8 to 17.5 (AP50 32.9→45.7), and removing the competing "person sitting"/"cyclist" prompts adds only 0.5 more (`results/metrics/diagnostics/yoloworld_prompts.txt`). "Person" is much more common than "pedestrian" in image–text training data, so it is plausibly a better-aligned text embedding. The reported results keep the original prompts, since choosing prompts on the evaluation set would be tuning on test data. Open vocabulary therefore buys flexibility and robustness for familiar objects, but not accuracy on domain-specific categories (cyclist, tram).

6.4 Fine-tuning vs zero-shot

YOLO11s on KITTI	Zero-shot	Fine-tuned
3-cls AP / AP50 / AP75	24.6 / 48.2 / 22.5	71.6 / 93.0 / 80.3
person / car / truck AP	19.5 / 45.3 / 9.0	49.8 / 80.9 / 83.9
AP_S / AP_L	14.0 / 36.8	58.3 / 80.7
7-class native AP	15.3 (YOLO-World)	67.2
Missed GT @conf 0.3	1,862	484
Background FPs @conf 0.3	1,182	281

Table 8. Fine-tuning (`results/tables/finetune.md`, `results/failures/*/summary.json`). Native per-class AP of the fine-tuned model: car 80.9, van 77.1, truck 83.9, pedestrian 49.7, person_sitting 43.3, cyclist 60.4, tram 74.9.

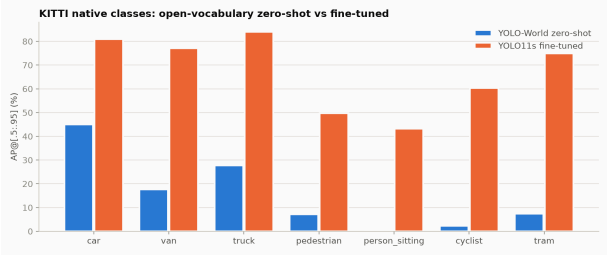


Figure 4. Native 7-class AP: open-vocabulary zero-shot vs fine-tuned.

25 minutes of fine-tuning almost triples KITTI AP. The *same* 9.4M-parameter network goes from the worst zero-shot model to far ahead of every other model. Truck jumps from 9.0 to 83.9, and AP75 from 22.5 to 80.3. The zero-shot gap is therefore mainly a **mismatch of label definitions and box conventions** (what counts as a truck vs a van, how tightly a pedestrian is boxed), not a lack of model capacity. Architecture choices moved KITTI AP by 4.4 points, whereas in-domain data moved it by 47. **Caveat:** train and val share no files, but KITTI frames are sampled from driving sequences, so frames of the same drive very likely appear in both splits. 71.6 is therefore an optimistic estimate compared with a sequence-level split.

7. Failure analysis

Each prediction (conf ≥ 0.3) and GT box was classified with a light TIDE-style [12] taxonomy at IoU 0.5: TP, FP-duplicate, FP-class (confusion), FP-loc (0.1≤IoU<0.5), FP-background (IoU<0.1) and FN (missed), with predictions on ignore regions skipped (Table 9).

Run (KITTI val, conf 0.3)	TP	FN (missed)	FP-background	FP-loc	FP-class	FP-dup	Recall person/small	Recall truck/medium
YOLO11s zero-shot	4,998	1,862	1,182	760	55	134	0.14	0.24
RT-DETR-L zero-shot	5,639	1,221	4,032	1,486	412	188	0.27	0.52
Faster R-CNN zero-shot	5,728	1,132	8,193	2,179	132	119	0.46	0.44
YOLO11s fine-tuned	6,376	484	281	315	4	173	0.46	0.97

Table 9. Error breakdown over 6,860 scored GT boxes (`results/failures/*/summary.json`). A fixed conf of 0.3 is a different operating point for each model, because score calibration differs. Compare counts within a model, not across models.

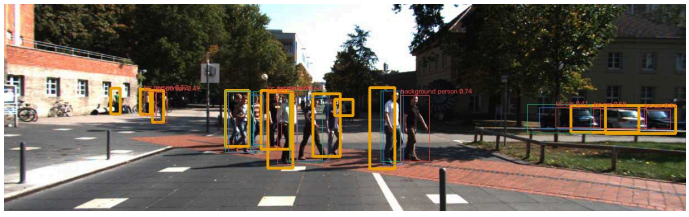


Figure 5a. YOLO11s, image 000535. A clearly visible pedestrian scored 0.74 is counted as *FP-background* because KITTI did not label him. The crowded group yields misses (thick orange) and loose boxes (FP-loc).



Figure 5b. Faster R-CNN, image 000845. Most red "background" FPs on the right are real parked cars behind a hedge and a person with a cart, all unlabelled. Heavily occluded parked cars on the left are missed.

What fails. (i) *Misses dominate*: small and occluded pedestrians (zero-shot YOLO11s finds 14% of small persons) and trucks (1 of 34 small trucks). Fine-tuning recovers most cars and trucks, but small pedestrians remain the hardest case even after training (46%). (ii) *Class confusion is rare* (55–412 boxes), and the main confusion is van→truck (Section 6.1). (iii) *"Background" false positives are partly a ground-truth artefact.* Without DontCare regions, real but unlabelled distant or

occluded objects are scored as hallucinations (Figure 5), so zero-shot precision is somewhat underestimated for every model. (iv) *Rows of overlapping parked cars* are a recurring miss pattern, consistent with NMS suppressing true neighbours.

8. Validating the harness: three problems caught by checks

- **FCOS failed the reproduction check** (31.3 vs 39.2 published). A controlled A/B on COCO isolated FP16 autocast as the cause: thresholds changed AP by <0.3, but FP32 restored 39.3. The loss is confined to the 77 classes not used on KITTI. Person, car and truck AP were unchanged (53.4/42.2/34.3 FP16 vs 53.5/42.4/34.3 FP32), while the other-77 mean fell from 39.2 to 30.8. This is why KITTI AP was unaffected (`results/metrics/diagnostics/fcos_fp16_per_class.txt`). FCOS therefore runs in FP32. A mean over 80 classes can hide where an error lives, and per-class checks expose it.
- **Latency outliers.** The first benchmark reported YOLO11s at 56.5 ms on COCO (median 5.4). Nine of 200 images carried a one-time per-input-shape setup cost of up to 1,046 ms, which disappeared on a second pass. The fix was to warm up on all 200 images.
- **Run-to-run noise.** One entire timing pass ran about 60% slow on Windows (8.3 vs 5.0 ms in three standalone repeats). Latency is now reported as the median of three passes with its range, rather than re-running until a number looks right.

9. Limitations

(1) No DontCare regions in the KITTI release, so precision is underestimated (Section 7). (2) COCO's absolute area thresholds are applied to 1242×375 images, which makes "small/medium/large" dataset-relative. (3) Truck AP rests on 223 boxes and person on 921. The 3-class mean weights truck equally with car's 5,716, so the KITTI ranking differences of 1–2 AP are within noise; no confidence intervals were computed. (4) One image-level train/val split, likely correlated by drive. (5) The taxonomy mapping is a design choice, so Table 5 reports how much it matters. (6) Latency was measured on Windows/WDDM with each framework's own preprocessing, and FCOS ran in FP32. (7) One fine-tuning run (seed 0), and only the smallest YOLO size was fine-tuned.

10. Conclusions

1. **COCO skill transfers unevenly.** On the shared classes, KITTI AP is 13–24 points lower, but car transfers almost perfectly while person and truck collapse. Reporting only mAP would hide this.
2. **The gap is mostly about definitions, not pixels or architecture.** Large objects degrade as much as small ones, higher resolution recovers only 2.3 AP, and the five architectures land within 4.4 AP of each other. Label conventions (van vs truck, pedestrian boxes), plus missing labels, explain more than the choice of detector.
3. **COCO rankings are a poor guide out of domain.** The worst COCO model (YOLO-World) ties for best on KITTI, and a mid-pack COCO model (YOLO11s) is last.
4. **In-domain data dominates.** Twenty-five minutes of fine-tuning (24.6→71.6) is worth roughly ten times the spread between architectures. For a robot, the practical recipe is a fast detector (5 ms) fine-tuned on the target domain, with per-class and per-scale evaluation, especially recall on small pedestrians, which remains the weakest point even after fine-tuning.
5. **Rigor matters.** Reproducing known numbers first, gating results with automated checks, and testing hypotheses with targeted experiments caught one wrong model setting and two measurement artefacts. It also refuted one of the study's own explanations: YOLO-World's "van" prompt does not explain its truck lead.

References

1. T.-Y. Lin, M. Maire, S. Belongie, et al. Microsoft COCO: Common Objects in Context. *ECCV*, 2014.
2. A. Geiger, P. Lenz, R. Urtasun. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. *CVPR*, 2012.
3. S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS*, 2015.
4. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie. Feature Pyramid Networks for Object Detection. *CVPR*, 2017.
5. Y. Li, S. Xie, X. Chen, P. Dollár, K. He, R. Girshick. Benchmarking Detection Transfer Learning with Vision Transformers. *arXiv:2111.11429*, 2021 (source of the Faster R-CNN v2 recipe).
6. Z. Tian, C. Shen, H. Chen, T. He. FCOS: Fully Convolutional One-Stage Object Detection. *ICCV*, 2019.

7. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, S. Zagoruyko. End-to-End Object Detection with Transformers. *ECCV*, 2020.
8. Y. Zhao, W. Lv, S. Xu, et al. DETRs Beat YOLOs on Real-time Object Detection (RT-DETR). *CVPR*, 2024.
9. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang, Y. Shan. YOLO-World: Real-Time Open-Vocabulary Object Detection. *CVPR*, 2024.
10. A. Radford, J. W. Kim, C. Hallacy, et al. Learning Transferable Visual Models From Natural Language Supervision (CLIP). *ICML*, 2021.
11. J. Redmon, S. Divvala, R. Girshick, A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. *CVPR*, 2016.
12. D. Bolya, S. Foley, J. Hays, J. Hoffman. TIDE: A General Toolbox for Identifying Object Detection Errors. *ECCV*, 2020.
13. Ultralytics. YOLO11 documentation (COCO performance table). https://docs.ultralytics.com/models/yolo11/
14. Ultralytics. RT-DETR documentation. https://docs.ultralytics.com/models/rtdetr/
15. Ultralytics. YOLO-World documentation (zero-shot transfer on COCO). https://docs.ultralytics.com/models/yolo-world/
16. PyTorch. torchvision.models: object detection weights table. https://docs.pytorch.org/vision/stable/models.html
17. COCO API (pycocotools), official repository. https://github.com/cocodataset/cocoapi
18. Ultralytics. KITTI dataset documentation. https://docs.ultralytics.com/datasets/detect/kitti/

Reproducibility and AI-tool use. Every number in this report is read from files that the pipeline wrote to `results/`. Sanity checks (`scripts/sanity_checks.py`) passed with 109 PASS, 0 WARN and 0 FAIL, and the command log is `commands.md`, with a stage-by-stage lab notebook in `docs/lab_notebook.md`. Claude Code (Anthropic) was used, as the assignment permits, to help write and debug the pipeline, run the experiments, and draft this report. Published numbers appear only as references, each checked against its official source, and the author reviewed the analysis against the results.